

HARMONY: A Hybrid Memory–Reasoning Architecture for Efficient Long-Context Intelligence

Installment 1: Motivation, Related Work, Design Goals, and Core Architecture

Abstract

Transformers became the dominant neural architecture because self-attention enabled strong sequence modeling with high parallelism during training. The original Transformer replaces recurrence and convolution with attention, and it remains the basis of most modern large language models. However, dense self-attention has quadratic cost in sequence length, and modern research has repeatedly explored ways to reduce that cost while preserving content-based reasoning.

This paper proposes **HARMONY: Hierarchical Adaptive Retrieval, Memory, and Online Neural Yielding network**. HARMONY is not a single replacement layer for attention. It is a **system architecture** that separates the functions that Transformers currently conflate: local syntax processing, long-range memory, factual retrieval, multi-hop reasoning, and answer verification. The design combines semantic compression, a selective state backbone, hierarchical retrieval memory, graph memory, sparse expert routing, and a planner-verifier loop.

The central hypothesis is that a better successor to Transformers will not simply “remove attention,” but will **redistribute computation** across specialized modules so that only the tasks that truly need global access pay for it.

1. Introduction

The Transformer architecture was introduced in 2017 as a simple sequence transduction model based solely on attention mechanisms, dispensing with recurrence and convolutions. It delivered strong results and better parallelization than earlier recurrent systems.

But the same design choice that made Transformers elegant also made them expensive: dense self-attention compares every token against every other token, which creates quadratic scaling in sequence length. This cost becomes a major obstacle when we want long documents, persistent memory, multi-document reasoning, or efficient inference at very large context windows. Recent alternative architectures were explicitly created to address this limitation: Mamba emphasizes linear-time selective state-space modeling, RetNet targets training parallelism with low-cost $O(1)$ inference, and Hyena proposes subquadratic long convolution operators for long sequences.

A second limitation is that Transformers store most knowledge in their weights. This makes the model powerful, but it also makes knowledge updates expensive and can weaken provenance and freshness. Retrieval-Augmented Generation was introduced precisely to combine a parametric generator with a non-parametric memory, improving knowledge-intensive NLP and making factual access more flexible. Dense Passage Retrieval likewise showed that semantic retrieval from embeddings can outperform traditional sparse retrieval methods such as BM25 in open-domain QA.

A third limitation is that “reasoning” in standard LLMs is largely implicit. The model produces the next token, but does not explicitly separate planning, evidence retrieval, and verification. HARMONY addresses this by making these processes explicit modules.

2. Related Work and What It Teaches Us

2.1 Transformer

The Transformer established that attention can replace recurrence and convolution for sequence modeling, while preserving parallelism. Its core mechanism is scaled dot-product attention, where each token attends to all other tokens through a learned similarity matrix. This works very well, but the matrix grows with the square of the sequence length.

2.2 Mamba

Mamba is important because it identifies a key weakness of many subquadratic sequence models: poor content-based reasoning. It introduces selective state-space modeling, where the model parameters are functions of the input, allowing selective propagation or forgetting of information. The authors report linear scaling, strong language results, and higher throughput than Transformers.

2.3 RetNet

RetNet shows that a retention mechanism can support three computation modes: parallel, recurrent, and chunkwise recurrent. The recurrent form enables $O(1)$ inference, while chunkwise recurrence supports efficient long-sequence modeling. This is directly relevant to any architecture that wants fast decoding and low memory use.

2.4 Hyena

Hyena demonstrates that dense attention is not the only path to strong long-context performance. It uses implicit long convolutions and gating, and the paper reports strong performance on long-context recall and reasoning tasks, including very large speedups at long lengths.

2.5 Retrieval-Augmented Generation and Dense Retrieval

RAG combines a pretrained parametric model with an external non-parametric index, solving part of the knowledge freshness and provenance problem. The original paper explicitly notes that parametric LMs have limited ability to access and precisely manipulate knowledge, while retrieval-based memory can overcome that limitation. DPR supports the retrieval side by showing that learned dense retrieval can outperform BM25 in open-domain QA.

2.6 Design lesson

The lesson from this literature is not that one mechanism is universally best. The lesson is that **different cognitive functions want different computational primitives**:

- local pattern extraction,
- long-range state tracking,
- factual memory access,
- multi-hop relational reasoning,
- and answer verification.

HARMONY is built on that lesson.

3. Design Goals

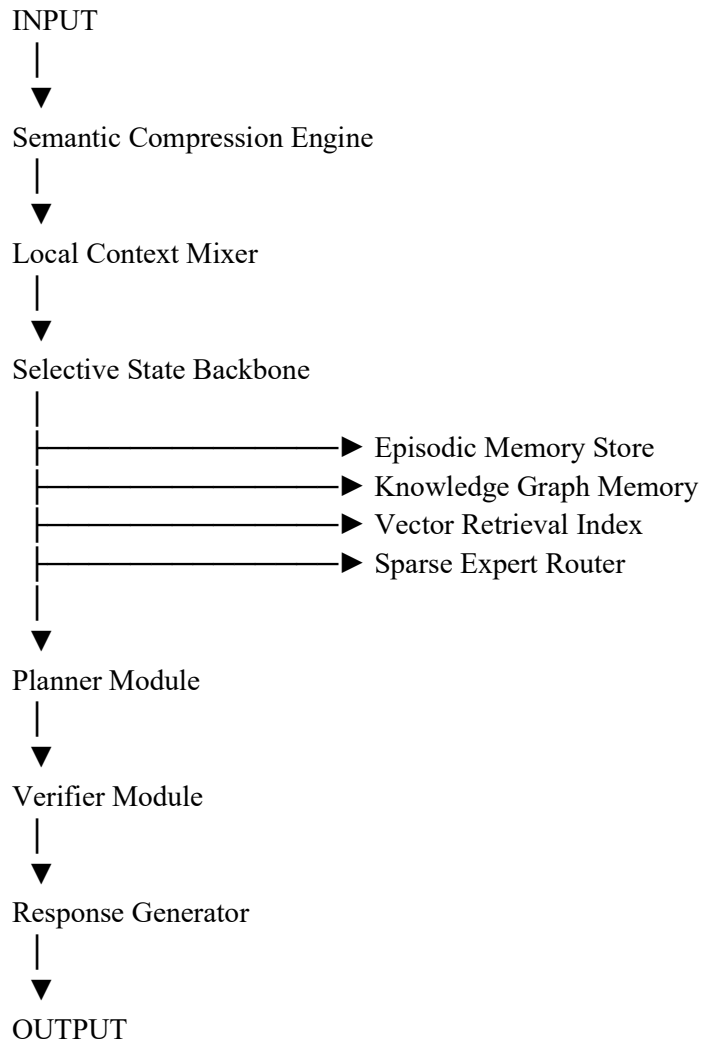
HARMONY is designed around six explicit goals:

1. **Lower memory usage** than dense Transformer attention for long inputs.
2. **Better long-range dependency handling** than pure sliding-window or pure local models.
3. **Better factual grounding** than parametric-only generation.
4. **Faster inference** through recurrent or chunkwise computation.
5. **Explicit reasoning control** through planner and verifier modules.
6. **Upgradeable knowledge** through external memory rather than retraining everything.

These goals are not independent. Lower memory allows longer context. Better retrieval improves factuality. Explicit planning improves multi-step reasoning. Together they create a system that is more suitable for document intelligence, assistants, code reasoning, scientific QA, and agentic workflows.

4. High-Level Architecture of HARMONY

4.1 Overview



4.2 Core idea

Instead of processing the entire input with one global attention layer, HARMONY decomposes the computation into four levels:

- **Level 1: Local understanding**
Parse short-range syntax, entities, and phrase-level structure.
- **Level 2: Sequential state**
Maintain a compact latent memory over long spans.
- **Level 3: External memory**
Retrieve facts, documents, and graph relations from storage.
- **Level 4: Reasoning control**
Plan the answer and verify that it is supported.

This is the key architectural difference from the Transformer. The Transformer uses a single universal mechanism; HARMONY uses a **composed cognitive stack**.

5. Module-by-Module Architecture

5.1 Semantic Compression Engine

Purpose

Raw token streams are noisy and often redundant. Many tokens exist only to support local grammar, not to carry semantic content. HARMONY therefore begins with a semantic compression stage that converts token sequences into more meaningful units.

Inputs

- raw token embeddings,
- positional information,
- optional entity tags,
- optional structural markers.

Outputs

- semantic chunks,
- entity summaries,
- clause summaries,
- code blocks,
- table fragments,
- graph nodes.

Conceptual operation

For each input span $x_{1:n}$, the compressor produces compressed semantic states:

$$c_i = f_{\text{comp}}(x_{s_i:e_i})$$

where each c_i represents a semantically coherent unit.

Why this matters

If a 2,000-token page can be compressed into 300 meaningful units, the backbone and retrieval layers process far less data. That does not just save memory; it makes reasoning cleaner because the model sees “thought units” instead of raw surface text.

Comparison with Transformers

Transformers can learn implicit compression through attention heads, but they still pay global cost on all tokens. HARMONY makes compression explicit before expensive reasoning begins.

5.2 Local Context Mixer

Purpose

Local syntax, formatting, nearby references, and short phrase dependencies should be modeled cheaply.

Design

Use a short-range mixer consisting of:

- sliding window attention,
- convolutional filtering,
- gated residual transforms.

Function

Given chunk embeddings c_i , the local mixer produces refined local states:

$$\ell_i = f_{\text{local}}(c_{i-w:i+w})$$

where w is a small window size.

Why this helps

Most token interactions in language are local. Only a fraction require global access. The local mixer handles the majority of easy work without invoking costly global reasoning.

Comparison with Transformers

Dense self-attention does local and global work together, which is simple but inefficient. HARMONY separates them.

5.3 Selective State Backbone

Purpose

This is the main long-range sequence backbone. It carries information across the document stream without constructing a full attention matrix.

Design inspiration

This component is inspired by the general direction of selective state-space models, RetNet-style retention, and other efficient sequence operators. Mamba shows that selective parameterization can recover content-based reasoning while maintaining linear scaling. RetNet shows a retention mechanism can support parallel, recurrent, and chunkwise computation, with $O(1)$ recurrent inference.

State update

A conceptual update rule is:

$$h_t = \alpha_t \odot h_{t-1} + \beta_t \odot \phi(x_t)$$

where:

- h_t is the latent state,
- α_t controls retention,
- β_t controls write strength,
- $\phi(x_t)$ projects the input into state space.

Interpretation

- If α_t is large, the system remembers.
- If β_t is large, the system writes new information.
- If both are small, the model ignores the token.

Why it improves long-range dependency

Attention retrieves long-range information by direct matching. State models instead carry long-range information forward through learned memory dynamics. This makes them naturally suited to streaming, long-document summarization, and incremental reasoning.

What HARMONY adds beyond existing state models

HARMONY does not rely on the state backbone alone. It combines state tracking with retrieval and verification, which is crucial because Mamba itself emphasizes that some sequence models struggle with content-based reasoning.

5.4 External Memory System

Purpose

Not all knowledge should live in weights. HARMONY stores knowledge in external memory, allowing explicit updates, provenance, and retrieval.

Memory types

1. **Episodic memory**
Past interactions, tasks, conversations, and user-specific context.
2. **Semantic memory**
Stable facts, definitions, concept summaries, and distilled knowledge.
3. **Working memory**
Temporary scratchpad-like storage for the current task.
4. **Vector memory**
Embedding index for semantic search.
5. **Graph memory**
Entities and relations for multi-hop reasoning.

Why external memory matters

RAG was introduced because pretrained LMs have limited ability to access and precisely manipulate knowledge, and because provenance and world-knowledge updating are open problems. RAG combines a parametric model with non-parametric memory to improve knowledge-intensive NLP.

Retrieval mechanism

For query state q and memory key m_i , define similarity:

$$s_i = \frac{q^\top m_i}{\|q\| \|m_i\|}$$

Top- k memory items are retrieved, reranked, and injected into the reasoning pipeline.

5.5 Hierarchical Retrieval Layer

Purpose

Flat chunk retrieval often fails on long documents because chunks may be too small, too noisy, or too isolated. HARMONY uses hierarchical retrieval.

Retrieval levels

- document level,
- section level,
- paragraph level,
- sentence level,
- entity level,
- relation level.

Why hierarchical retrieval is better

A broad document-level pass can filter irrelevant sources, then finer retrieval can zoom in to exact evidence. This is especially helpful for legal documents, research papers, technical manuals, and multi-hop QA. Dense Passage Retrieval showed that learned retrieval can outperform BM25 for open-domain QA, which supports the idea that retrieval should be learned rather than purely lexical.

5.6 Graph Memory Subsystem

Purpose

Some questions are not just about nearby text; they are about relational chains:

- who is connected to whom,
- what caused what,
- what belongs to what,
- what was derived from what.

A graph memory makes these relations explicit.

Representation

Nodes represent entities or concepts. Edges represent relations.

$$G = (V, E)$$

where:

- V = entity nodes,
- E = typed relations.

Why it matters

Vector retrieval is excellent for fuzzy semantic matching, but graph memory is better for structured multi-hop traversal. This is important for scientific QA, enterprise knowledge bases, and reasoning over linked facts.

5.7 Sparse Expert Router

Purpose

Different inputs need different skills. Code, math, dialogue, biomedical text, legal text, and planning do not use identical transformations.

Design

A router computes expert probabilities:

$$p(e_i | x) = \text{Softmax}(W_r x)$$

Then only the top experts are activated.

Why this helps

- More capacity without full dense compute.
- Specialization by task domain.
- Lower inference cost than activating every parameter.

Relationship to the rest of HARMONY

The router sits above the backbone and memory layers, selecting whether the current input should be handled by a math expert, code expert, factual expert, or planning expert.

5.8 Planner Module

Purpose

The planner decides **what to do before answering**.

Example flow

For a difficult query:

1. parse question,
2. identify missing information,
3. decide retrieval strategy,
4. determine whether graph traversal is needed,
5. choose whether expert routing is required,

6. produce a reasoning plan.

Why it matters

Most modern LLMs implicitly “plan” through token generation. HARMONY makes planning explicit, which improves control and reduces aimless generation.

5.9 Verifier Module

Purpose

The verifier checks whether the answer is supported by evidence.

Checks performed

- factual support,
- contradiction detection,
- retrieval relevance,
- consistency across claims,
- confidence estimation.

Output

- accept answer,
- revise answer,
- retrieve more evidence,
- or abstain.

Why it matters

RAG improves grounding, but it does not automatically solve hallucination. The RAG literature itself notes open problems in provenance and updated world knowledge. Verification is the missing control layer.

6. Why HARMONY should outperform a Transformer in the dimensions that matter

6.1 Memory usage

Transformer memory grows heavily with sequence length because attention tracks pairwise interactions. HARMONY reduces this by:

- compressing before processing,
- using a recurrent/selective state backbone,
- retrieving only relevant memory,
- activating only a subset of experts.

6.2 Long-range dependency

Instead of storing long-range context only in attention scores, HARMONY maintains persistent state and external memory.

6.3 Context learning

By combining retrieval with semantic compression, the model can learn from long documents more efficiently than a flat token processor.

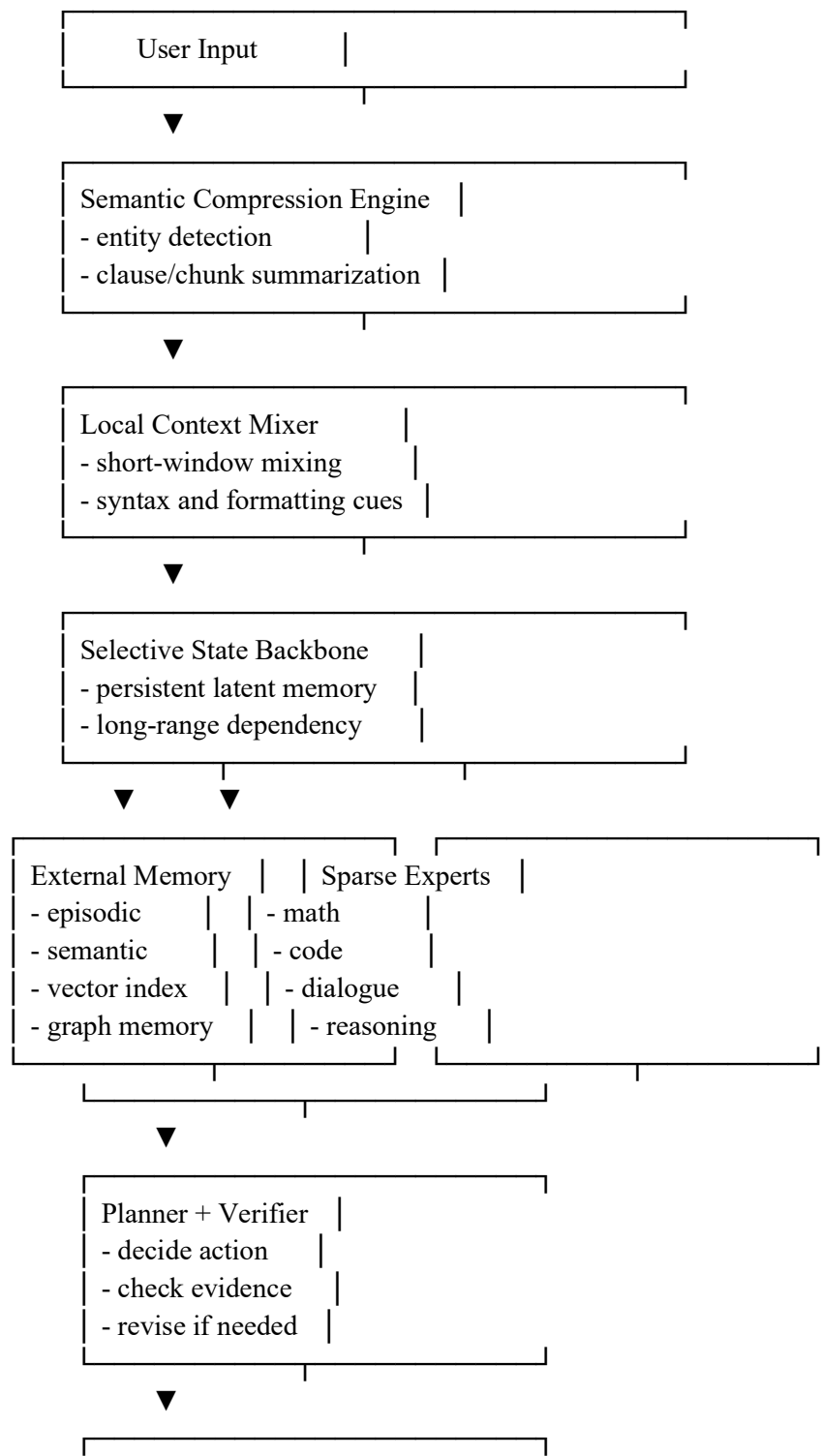
6.4 Factual grounding

External memory makes facts explicit and updatable, unlike purely parametric memory. This is one of the main motivations of RAG.

6.5 Inference speed

Mamba reports higher throughput than Transformers and linear scaling, RetNet reports $O(1)$ inference in recurrent mode, and Hyena reports substantial speedups at long lengths. HARMONY aims to inherit the strengths of these directions while adding retrieval and verification.

7. Visual summary of the HARMONY workflow



9. Mathematical Foundations of HARMONY

9.1 Formal Problem Definition

Given:

$$X = \{x_1, x_2, x_3, \dots, x_n\}$$

where:

- x_i represents an input token,
- n represents sequence length.

The objective is:

$$P(Y | X, M, G, E)$$

Where:

- Y = generated output
- X = input sequence
- M = external memory
- G = knowledge graph
- E = expert subnetworks

Unlike Transformers:

$$P(Y | X)$$

HARMONY becomes:

$$P(Y | X, M, G, E, P, V)$$

Where:

- P = planner
- V = verifier

Thus intelligence becomes a composition of subsystems.

9.2 Information Processing Pipeline

The complete information transformation is:

Input



Semantic Compression



State Backbone



Memory Retrieval



Graph Reasoning



Expert Routing

↓

Planning

↓

Verification

↓

Generation

Formally:

$$Y = f_{gen}(f_{verify}(f_{plan}(f_{expert}(f_{graph}(f_{memory}(f_{state}(f_{compress}(X))))))))))$$

9.3 Semantic Compression Mathematics

Motivation

Language contains redundancy.

Example:

The capital city of France is Paris.

Contains:

Entity: France

Relation: Capital

Value: Paris

Many tokens contribute little semantic value.

Compression Function

Input:

$$X = \{x_1, x_2, \dots, x_n\}$$

Compression:

$$C = f_c(X)$$

where:

$$C = \{c_1, c_2, \dots, c_m\}$$

and:

$$m \ll n$$

Example:

2000 tokens

↓

250 semantic units

Compression Ratio:

$$CR = \frac{n}{m}$$

Example:

$$CR = \frac{2000}{250} = 8$$

8x compression.

Information Preservation Objective

We maximize:

$$I(C, X)$$

where:

I = Mutual Information

Meaning:

Compressed representation should preserve maximum information.

9.4 Selective State Backbone

Core Concept

Instead of attention matrices:

Store evolving memory state.

State:

$$h_t$$

Standard State Update

$$h_t = A_t h_{t-1} + B_t x_t$$

Where:

- A_t controls retention
- B_t controls write operations

Learnable Memory Gate

Forget gate:

$$f_t = \sigma(W_f x_t + b_f)$$

Write gate:

$$w_t = \sigma(W_w x_t + b_w)$$

New state:

$$h_t = f_t \odot h_{t-1} + w_t \odot x_t$$

Memory Importance Gate

Every semantic chunk receives importance score:

$$s_t = \sigma(W_s x_t)$$

Only high-scoring information enters long-term memory.

Why Better than Attention?

Transformer:

Every token attends to every token.

Complexity:

$$O(n^2)$$

State Backbone:

$$O(n)$$

9.5 Hierarchical Memory System

Human Memory Inspiration

Humans do not remember everything equally.

Memory hierarchy:

Working Memory

↓

Episodic Memory

↓

Semantic Memory

HARMONY mimics this.

9.6 Working Memory

Stores:

Current task context.

Representation:

$$W = \{w_1, w_2, \dots, w_k\}$$

Very small.

Fast access.

9.7 Episodic Memory

Stores:

Experiences.

Structure:

$$e_i = (\text{content}, \text{timestamp}, \text{importance})$$

Episodic Retrieval

Similarity:

$$\text{sim}(q, e_i)$$

Top-k:

$$E^* = \text{TopK}(\text{sim}(q, e_i))$$

9.8 Semantic Memory

Stores distilled knowledge.

Example:

Earth $\rightarrow$ Planet

Paris $\rightarrow$ Capital of France

Permanent memory.

9.9 Knowledge Graph Mathematics

Graph:

$$G = (V, E)$$

Where:

V =nodes

E =edges

Example:

Paris

|

CapitalOf

|

France

Multi-Hop Traversal

Question:

Capital of country where Eiffel Tower exists

Traversal:

$$\text{EiffelTower} \rightarrow \text{Paris} \rightarrow \text{France}$$

Graph Score

$$\text{Score}(\text{path}) = \sum \text{edge_weights}$$

Best path selected.

9.10 Hybrid Memory Fusion

Final Memory Context:

$$M = \alpha E + \beta S + \gamma G$$

Where:

- E = episodic memory
- S = semantic memory
- G = graph memory

Weights learned dynamically.

10. Sparse Expert Routing

Motivation

Not every query requires same expertise.

Example:

Math:

2+2

Needs different computation than:

Write a poem

Expert Set

$$E = \{e_1, e_2, e_3, \dots, e_n\}$$

Examples:

Math Expert

Code Expert

Reasoning Expert

Vision Expert

Dialogue Expert

Router

$$p_i = \text{Softmax}(W_r x)$$

Top-K Selection

Select:

$$\text{TopK}(p_i)$$

Only those experts activate.

Computational Savings

Transformer:

100% network active

HARMONY:

5-10% experts active

Massive FLOP reduction.

11. Planner Architecture

Current LLM:

Question

↓

Answer

HARMONY:

Question

↓

Plan

↓

Retrieve

↓

Reason

↓

Verify

↓

Answer

Planner State

p_t

Planner outputs:

$Action = f(p_t)$

Possible actions:

Retrieve

Graph Search

Math Expert

Code Expert

Generate

11.1 Task Decomposition

Example:

Who invented relativity and where was he born?

Planner:

Step 1:

Find inventor

Step 2:

Retrieve birthplace

Step 3:

Combine answers

This converts reasoning into explicit operations.

12. Verifier Architecture

Purpose

Prevent hallucination.

Verification Function

Input:

Answer

Evidence

Memory

Output:

Confidence

Confidence Score

$$C = f(Evidence, Consistency, Support)$$

Hallucination Detection

If:

$$C < Threshold$$

Then:

Reject Answer

Retrieve Again

12.1 Contradiction Detection

Example:

Claim:

Earth has 3 moons

Memory:

Earth has 1 moon

Verifier flags contradiction.

13. Generator Module

Generator receives:

$$Z = [State, Memory, Graph, Experts, Plan]$$

Final generation:

$$Y = f_{gen}(Z)$$

Unlike GPT:
Generator is final stage.
Not the entire intelligence system.

14. Unified Loss Function

Current LLM:

$$L = L_{LM}$$

Only next-token prediction.

HARMONY:
Multiple objectives.

Language Loss

$$L_{LM}$$

Retrieval Loss

$$L_{RET}$$

Measures retrieval quality.

Planning Loss

$$L_{PLAN}$$

Measures planning correctness.

Verification Loss

$$L_{VER}$$

Measures factual consistency.

Memory Loss

$$L_{MEM}$$

Measures memory utilization.

Graph Loss

$$L_{GRAPH}$$

Measures graph traversal accuracy.

Complete Objective

$$L = \alpha L_{LM} + \beta L_{RET} + \gamma L_{PLAN} + \delta L_{VER} + \epsilon L_{MEM} + \zeta L_{GRAPH}$$

This becomes the master optimization objective.

15. Training Pipeline

Stage 1

Semantic Compression Pretraining

Learn:

Token $\rightarrow$ Concept

Stage 2

State Backbone Training

Learn:

Long-range dependencies

Stage 3

Memory Training

Learn:

Store

Retrieve

Update

Stage 4

Graph Training

Learn:

Relations

Traversal

Reasoning

Stage 5

Expert Specialization

Train experts independently.

Stage 6

Planner Training

Teach decomposition.

Stage 7

Verifier Training

Teach fact checking.

Stage 8

Joint Optimization
Train all modules together.

16. Complexity Analysis

Transformer

Memory:

$$O(n^2)$$

Compute:

$$O(n^2)$$

HARMONY

Compression:

$$O(n)$$

State Backbone:

$$O(n)$$

Retrieval:

$$O(\log N)$$

Graph Search:

$$O(k)$$

Expert Routing:

$$O(k)$$

Approximate Total:

$$O(n)$$

for sequence modeling.

17. Potential Failure Modes

Memory Pollution

Bad information stored.

Solution:

Memory quality scoring.

Retrieval Drift

Wrong documents retrieved.

Solution:
Multi-stage reranking.

Expert Collapse

One expert receives all traffic.

Solution:
Load balancing loss.

Graph Explosion

Knowledge graph becomes huge.

Solution:
Hierarchical graph partitioning.

Planner Failure

Wrong reasoning plan.

Solution:
Planner self-reflection loop.

18. Key Innovation Summary

HARMONY replaces:

One Giant Attention Mechanism
with:

Compression

+

State Memory

+

Retrieval

+

Graph Reasoning

+

Experts

+

Planning

+

Verification

creating a cognitive architecture rather than a pure sequence model.

Neural Architecture, Tensor Flow Design, Memory Algorithms, Training Curriculum, and Compute Architecture

37. Transition from Conceptual Design to Neural Architecture

In Parts 1–3, HARMONY was described as a cognitive architecture.

Now we define:

- Neural components

- Tensor flow
- Internal representations
- Training mechanisms
- Data structures
- Compute requirements

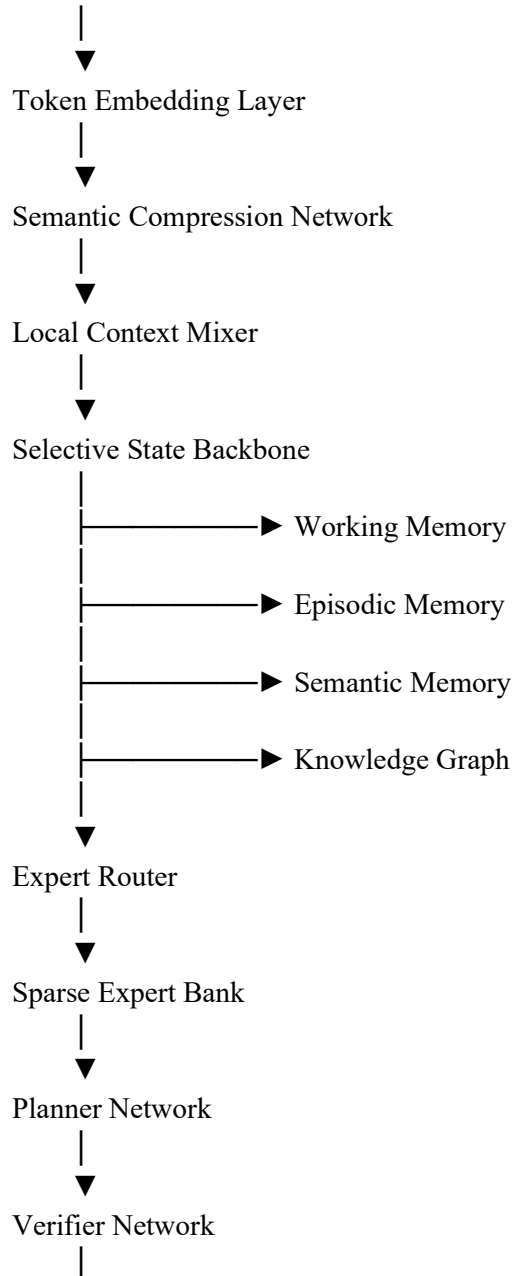
This section attempts to answer:

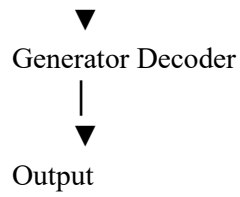
"If I were to build HARMONY tomorrow, what exactly would each module look like?"

38. Global Neural Architecture

High-Level Tensor Flow

Raw Input Tokens





39. Input Representation System

Transformer Problem

Transformers represent text as:

$$X \in \mathbb{R}^{n \times d}$$

where:

- n = sequence length
- d = embedding dimension

All tokens treated equally.

HARMONY Input Representation

Each token receives:

Token Embedding

+

Semantic Type Embedding

+

Entity Embedding

+

Structural Embedding

+

Temporal Embedding

Therefore:

$$x_i = e_t + e_s + e_e + e_p + e_{time}$$

Where:

- e_t = token embedding
- e_s = semantic embedding
- e_e = entity embedding

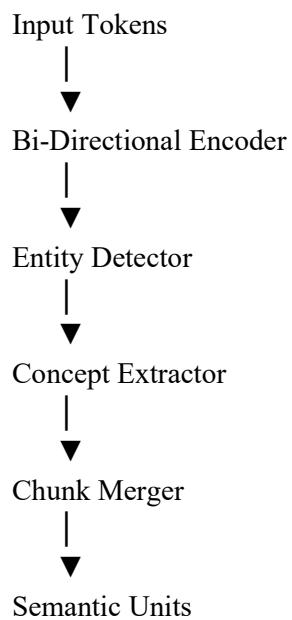
- e_p = positional embedding
- e_{time} = temporal embedding

40. Semantic Compression Network (SCN)

Goal

Transform:
Thousands of tokens
into:
Hundreds of semantic units

Architecture



Internal Representation

Output:

$$C = \{c_1, c_2, \dots, c_m\}$$

where:

$$m \ll n$$

Example

Input:
5000 tokens
Output:
600 semantic units

Semantic Compression Objective

Learn:

$$P(C | X)$$

while maximizing retained information.

Compression Loss

$$L_{compress} = 1 - I(C, X)$$

where:

I = mutual information.

41. Local Context Mixer (LCM)

Purpose:

Handle short-range dependencies.

Why?

Most language interactions are local.

Example:

The cat sat on the mat.

Dependencies exist within nearby words.

Global retrieval unnecessary.

Architecture

Depthwise Convolution

+

Gated Attention

+

Residual Block

Input:

$$C$$

Output:

$$L$$

Complexity

Window Size:

$$w$$

Complexity:

$$O(nw)$$

instead of:

$$O(n^2)$$

42. Selective State Backbone (SSB)

This is the heart of HARMONY.

State Representation

Each timestep:

$$h_t$$

contains:

Semantic State

Memory State

Reasoning State

Planning State

Combined:

$$h_t = [h_s, h_m, h_r, h_p]$$

42.1 Multi-Scale State

Instead of one state:

Use hierarchy.

Level 1

Token Memory

$$h_t^1$$

Level 2

Sentence Memory

$$h_t^2$$

Level 3

Document Memory

$$h_t^3$$

Level 4

Session Memory

$$h_t^4$$

Final state:

$$H_t = \{h_t^1, h_t^2, h_t^3, h_t^4\}$$

42.2 State Update Mechanism

Update:

$$h_t = g_f h_{t-1} + g_w x_t + g_r R_t$$

where:

- g_f = forget gate
- g_w = write gate
- g_r = retrieval gate

This introduces memory injection directly into state updates.

43. Working Memory Architecture

Equivalent to CPU cache.

Stores:

Current Task

Current Evidence

Current Plan

Current State

Memory Size:

Fixed.

Example:

2048 slots

Access:

$$O(1)$$

44. Episodic Memory Architecture

Stores experiences.

Memory Entry

$$M_i = (key, value, timestamp, importance)$$

Example:

User asked about Physics

Summary generated

Stored

44.1 Memory Write Algorithm

Write Score:

$$W_i = Importance \times Novelty \times Utility$$

Only high-score entries stored.

45. Semantic Memory Architecture

Stores compressed concepts.

Example:

Gravity

↓

Force

↓

Physics

Implemented as:

Concept Graph

+

Vector Index

46. Knowledge Graph Engine

Graph:

$$G = (V, E)$$

Node Example

Einstein

Edge:

Born In

Target:

Ulm

46.1 Graph Neural Layer

Each node:

$$v_i$$

updated via:

$$v'_i = \sum_{j \in N(i)} \alpha_{ij} W v_j$$

This allows relational reasoning.

47. Expert Router Architecture

Input:

Current Query

Memory State

Graph State

Planner State

Combined Representation:

$$z$$

Router:

$$p = \text{Softmax}(W_r z)$$

Select:

Top-K Experts

Example:

Math Query

↓

Math Expert
Reasoning Expert

48. Sparse Expert Network

Each expert:
Independent neural system.

Example
Math Expert:
Symbolic Manipulation

Theorem Retrieval

Equation Solving

Code Expert:
Code Understanding

Program Synthesis

Debugging

Science Expert:
Research Knowledge

Scientific Reasoning

49. Planner Neural Architecture

Planner receives:
State

Memory

Evidence

Produces:
Action Sequence

Architecture:
Mini Transformer

+

State Module

Output:

$$Plan = \{a_1, a_2, \dots, a_n\}$$

Example:

Retrieve

↓

Graph Search

↓

Reason

↓

Generate

50. Verifier Neural Architecture

Acts as internal critic.

Inputs:

Generated Answer

Evidence

Memory

Plan

Outputs:

Confidence

Correction Signal

Confidence:

$$C = \sigma(Wz)$$

If:

$$C < 0.7$$

restart reasoning.

51. Generator Architecture

Unlike GPT:
Generator is final stage.

Input:

$$Z = [State, Memory, Graph, Experts, Plan]$$

Output:

Y

Generator becomes:
Narrator

Not Thinker

52. Training Data Pipeline

Stage 1

Language Data

Sources:

Books

Wikipedia

Web Data

Research Papers

Stage 2

Reasoning Data

Sources:

Mathematics

Logic

Proofs

Algorithms

Stage 3

Planning Data

Sources:

Agent Tasks

Tool Use

Workflows

Stage 4

Verification Data

Sources:

Fact Checking

Contradiction Detection

53. Curriculum Learning Strategy

Train progressively.

Phase 1

Language

Phase 2

Reasoning

Phase 3

Memory

Phase 4

Planning

Phase 5

Verification

Phase 6

Joint Training

54. Synthetic Data Generation

Problem:

Human-generated reasoning data insufficient.

Solution:

Self-play.

Generate:

Question

Reasoning Trace

Verification Trace

Store.

Retrain.

55. Reinforcement Learning Layer

Reward:

$$R = Accuracy + Reasoning + Verification + MemoryUsage$$

Objective:

$$J(\theta) = E[R]$$

Model learns:

Better plans

Better retrieval

Better reasoning

56. Compute Requirements

Approximate HARMONY v1

Parameters:

70B

Memory:

10+ TB

External memory.

Training Tokens:

10T+

Compute:

10^{25} – 10^{26} FLOPs

Hypothetical frontier-scale estimate.

57. Why HARMONY Could Surpass Transformers

Transformers spend computation on:

Attention
everywhere.

HARMONY spends computation on:

Reasoning only when needed

Memory only when needed

Experts only when needed

Verification only when needed

This creates:

Adaptive Computation

instead of:

Uniform Computation